

Mini-Projects

Exercise: Build a simple chat interface to interact with a local model using the Ollama API.

Task:

Write a Python script to create a chat loop where the user can enter messages, and these messages are sent to a local model through the `ollama.chat()` function. Your script should keep track of the entire conversation history — both user inputs and model responses — and send this history with each query to ensure contextual understanding.

Requirements:

- Initialize an empty list to store the conversation messages.
- Define a function `chat(message)` that:
 - Adds the user's current message to the conversation history.
 - Sends the entire conversation history to the model using `ollama.chat()`.
 - Appends the model's response to the conversation history.
 - Prints the model's reply.
- Implement a loop that continuously prompts the user for input until they type `'exit'`, `'quit'`, or `'q'`.
- On each user input, call the `chat()` function with the message.

Optional: Create a streamlit UI for the chat.

Exercise: Building a RAG-based Question-Answering Pipeline with LangChain

Create a Retrieval-Augmented Generation (RAG) system using LangChain and a pdf file of your choice. By the end, the pipeline should answer questions about the PDF contents.

1. Load a PDF Document

- Load the PDF from a local directory with `PyPDFLoader`.
- Extract and inspect the documents to ensure the PDF was loaded correctly.

2. Preprocess & Chunk the Data

- Use `RecursiveCharacterTextSplitter` to break the text into manageable overlapping chunks.

3. Generate Embeddings

- Use Ollama's local MiniLM model (`all-minilm:33m`) to transform text chunks into vector representations.

4. Store Chunks in a Vector Database

- Build a Chroma vector store to enable efficient similarity-based searches on your document chunks.

5. Create the Prompt and System Message

- Set up a concise system prompt to instruct the language model to answer questions using the provided context only and keep responses short (no more than three sentences).

6. Load the LLM and Setup Retrieval

- Use a local LLM for answer generation.
- Configure the retriever to return the top-6 most relevant chunks per query.

7. Build and Run the RAG Chain

- Combine the retrieval and generation steps into a single pipeline.
- Pass the question through the RAG chain.

Exercise: Building a LangChain SQL Agent

Create a **LangChain SQL Agent** that can access a PostgreSQL database.

- Connect to a PostgreSQL database with LangChain
- Ask natural language questions and generate SQL queries
- Interpret agent responses

1. Database Connection

- Establish a connection to a PostgreSQL database.
- Confirm your connection by printing the SQL dialect and listing all usable tables.

2. Building the Agent

- Integrate a language model (e.g., ChatOllama/mistral) into LangChain.
- Create a SQL Database Toolkit with the database connection and LLM.

Hint: `toolkit = SQLDatabaseToolkit(db=db, llm=llm)`

- Retrieve and examine available toolkit tools.

3. Custom System Prompt and Agent Initialization

- Use the custom prompt (`SQL_PREFIX`) instructing the agent to:
 - Always check available tables first.
 - Only generate relevant queries (never select all columns).
 - Avoid DML statements (do not modify the database).
 - Limit query results unless otherwise specified.

4. Natural Language SQL Queries

- Use the agent to answer questions about the database